

Reinforcement Learning

30-Day, 300-Question Mastery Workbook

From Python Beginner to Practical Reinforcement Learning

A comprehensive problem-solving course designed to take you from RL foundations through deep reinforcement learning.

Generated: September 16, 2026

How to Use This Workbook

Structure

This workbook contains 30 days of structured learning, with 10 questions per day (300 questions total). Each day builds on previous concepts with progressive difficulty.

Daily Approach

1. Read the learning objectives for the day
2. Work through all 10 questions without looking ahead
3. Show all calculations and reasoning
4. Refer to the answer key only after completing your attempt
5. Review any questions you struggled with

Question Types

- **Beginner:** Foundation concepts, basic calculations
- **Intermediate:** Algorithm reasoning, multi-step problems
- **Advanced:** Deep synthesis, design problems, complex analysis

Prerequisites

You should know:

- Python programming (variables, loops, functions, lists, dictionaries)
- Basic algebra (solving for x , working with equations)
- How to run Python code

What You'll Learn

By Day 30, you will understand and be able to implement:

- Markov Decision Processes and value functions
- Monte Carlo, Temporal Difference, and Q-learning
- Exploration-exploitation trade-offs
- Deep Q-Networks and policy gradients
- Actor-Critic methods and PPO
- How to debug RL agents and design RL environments

30-Day Roadmap

Days	Topic	Focus
1-5	RL Foundations	Agent, environment, reward, policy, episode
6-10	MDPs & Value Functions	Bellman equations, dynamic programming, value iteration
11-15	Monte Carlo, TD, Q-Learning	TD learning, SARSA, Q-learning updates
16-20	Exploration & Implementation	Bandits, ϵ -greedy, tabular RL, GridWorld
21-25	Math & Deep Q-Learning	Probability, linear algebra, neural networks, DQN
26-28	Policy Gradients & Actor-Critic	REINFORCE, actor-critic, PPO
29	Algorithm Synthesis	Compare and reason about all algorithms
30	Final Mastery Assessment	Comprehensive synthesis and application

Table of Contents

How to Use This Workbook	2
30-Day Roadmap	3
Day 1	[Page TBD]
Day 2	[Page TBD]
Day 3	[Page TBD]
Day 4	[Page TBD]
Day 5	[Page TBD]
Day 6	[Page TBD]
Day 7	[Page TBD]
Day 8	[Page TBD]
Day 9	[Page TBD]
Day 10	[Page TBD]
Day 11	[Page TBD]
Day 12	[Page TBD]
Day 13	[Page TBD]
Day 14	[Page TBD]
Day 15	[Page TBD]
Day 16	[Page TBD]
Day 17	[Page TBD]
Day 18	[Page TBD]
Day 19	[Page TBD]
Day 20	[Page TBD]
Day 21	[Page TBD]
Day 22	[Page TBD]
Day 23	[Page TBD]
Day 24	[Page TBD]
Day 25	[Page TBD]
Day 26	[Page TBD]
Day 27	[Page TBD]
Day 28	[Page TBD]
Day 29	[Page TBD]
Day 30	[Page TBD]

Questions Workbook

Day 1 — RL Foundations: Agent, Environment, and Rewards

Learning Objectives

- Understand what Reinforcement Learning is and how it differs from other ML paradigms
- Learn core RL terminology: agent, environment, state, action, reward
- Understand episodes and timesteps in RL
- Begin thinking about reward design

Q1. ■ Describe a video game (e.g., Super Mario, chess, or an arcade game) as an RL problem. Who is the agent? What is the environment? What are some states, actions, and rewards?

Q2. ■ An agent is learning to navigate a maze. At each timestep, the agent perceives walls and paths around it (this is the state). It chooses to move up, down, left, or right (these are actions). Explain: what is the reward function? What would be a good reward design?

Q3. ■ In a robot arm reaching for a target, explain the difference between: (a) giving the robot +100 reward only when it reaches the target, and (b) giving the robot -1 reward at every timestep until it reaches the target (then +100). Which might lead to faster learning? Why?

Q4. ■ A self-driving car learns to navigate traffic. What are the challenges in designing a reward function for this task? Should you reward speed? Safety? Efficiency? How would you balance these?

Q5. ■ Define an "episode" in RL. Give two examples: one where an episode has a clear ending, and one where it might not.

Q6. ■ Why is Reinforcement Learning different from supervised learning? Explain at least two key differences using an example.

Q7. ■ An RL agent learns to trade stocks. The agent observes the stock price (state), decides to buy, hold, or sell (action), and receives profit/loss as reward. What might go wrong if you reward based only on short-term profits? How might you improve the reward function?

Q8. ■ What is a "timestep"? In a game of chess, how many timesteps might occur in one game? In a robot walking, how many timesteps might occur in one "step"?

Q9. ■ Explain why Reinforcement Learning is hard compared to supervised learning. What does the agent not know that makes learning more difficult?

Q10. ■ A recommendation system RL agent learns to recommend movies to users. The agent observes user profiles and viewing history (state), recommends a movie (action), and receives a reward based on whether the user watched and liked it. What is a challenge with this reward signal? (Hint: think about delays and feedback.)

Day 2 — Returns, Discounting, and Policies

Learning Objectives

- Understand return (cumulative reward)
- Learn the discount factor γ (gamma)
- Calculate discounted returns
- Understand deterministic and stochastic policies

Q11. ■ An agent receives rewards: +1, +2, +3 in three consecutive timesteps. With γ (discount factor) = 1, what is the total return? With $\gamma = 0.9$, what is the return? Show your calculation.

Q12. ■ Explain in your own words: what does the discount factor γ represent? Why might we want $\gamma < 1$ (e.g., $\gamma = 0.9$) instead of $\gamma = 1$?

Q13. ■ An agent receives rewards: +10, 0, 0, -5 over four timesteps. Calculate the return (cumulative discounted reward) from timestep 1 using $\gamma = 0.95$. Show all steps.

Q14. ■ What is a "policy" in Reinforcement Learning? Give an example of a policy in a game (e.g., chess or a simple video game).

Q15. ■ Explain the difference between a deterministic policy and a stochastic policy. Give an example of each in a real-world scenario.

Q16. ■ An agent chooses actions using the policy: $P(\text{up}) = 0.6$, $P(\text{down}) = 0.2$, $P(\text{left}) = 0.1$, $P(\text{right}) = 0.1$. Is this a deterministic or stochastic policy? Why might a stochastic policy be useful?

Q17. ■ Calculate the return from timestep 2 (not timestep 1) given rewards: $r_1=5$, $r_2=3$, $r_3=2$, $r_4=1$ using $\gamma = 0.9$. Show your calculation.

Q18. ■ If $\gamma = 0$, what does this mean for how the agent values future rewards? If $\gamma = 1$, what does it mean? Which is more realistic for a long-term task?

Q19. ■ An agent receives rewards: 100, 100, 100 every timestep indefinitely. With $\gamma = 0.9$, does the return converge to a finite value? Calculate the total return (infinite sum).

Q20. ■ In a Partially Observable Markov Decision Process (POMDP), the agent does not fully observe the state. How might this complicate learning a policy? Give an example.

Day 3 — Markov Property and Markov Decision Processes

Learning Objectives

- Understand the Markov property
- Learn the formal definition of an MDP
- Understand state transitions and transition probabilities
- Calculate transition probabilities

Q21. ■ State the Markov property in your own words. Why is it called "memoryless"?

Q22. ■ Is a chess game Markovian? That is, does the optimal next move depend only on the current board position, or does it depend on the entire history of moves? Explain.

Q23. ■ Define a Markov Decision Process (MDP). What are the five components of an MDP?

Q24. ■ In a gridworld, an agent at position (2,3) moves up with an intended action "up". Due to wind, the agent ends up at (2,2) with probability 0.8, or (1,3) with probability 0.2. Write the state transition probability: $P(s'|s, a)$ for these two outcomes.

Q25. ■ Consider a simple 2-state MDP: state A and state B. From state A, action "go" transitions to state B with probability 1. From state B, action "return" transitions back to state A with probability 1. Write out the transition matrix $P(s'|s, a)$.

Q26. ■ An agent is in state $s = \text{"sunny"}$. The reward function gives $R(s=\text{"sunny"}, a=\text{"outside"}) = +10$. Why is the Markov property important for RL algorithms?

Q27. ■ A robot arm is in a "bent" state. It takes action "extend". The probability of reaching "extended" state is 0.95, and probability of "semi-extended" state is 0.05. Write $P(s'|s=\text{"bent"}, a=\text{"extend"})$ for both outcomes.

Q28. ■ Suppose a patient in a medical treatment RL agent is in state "sick". The agent chooses action "give_medicine". With probability 0.7, the patient transitions to "recovering"; with probability 0.2, "same"; with probability 0.1, "worsening". Verify that these probabilities sum to 1 and write the transition probabilities.

Q29. ■ Is the weather Markovian? That is, is tomorrow's weather determined only by today's weather, or does it depend on the entire history? Explain and discuss implications for an RL weather prediction agent.

Q30. ■ In an MDP, define the reward function $R(s, a)$ and the transition probability $P(s'|s, a)$. Can the reward depend on the next state s' instead of just s and a ? Explain how this changes the MDP.

Day 4 — Trajectories, Episodes, and Environment Dynamics

Learning Objectives

- Understand trajectories as sequences of state-action-reward tuples
- Learn how environments generate trajectories
- Understand terminal states and episode boundaries
- Trace through episodes step-by-step

Q31. ■ Define a "trajectory" in RL. What does a trajectory look like? (Hint: it's a sequence of what?)

Q32. ■ An agent starts in state $s = \text{"start"}$. It takes action $a = \text{"go"}$, receives reward $r = 1$, and transitions to $s = \text{"middle"}$. Then it takes action $a = \text{"continue"}$, receives $r = 2$, and reaches $s = \text{"goal"}$ (a terminal state). Write out the full trajectory.

Q33. ■ What is a "terminal state"? Give two examples of terminal states in different RL environments.

Q34. ■ Trace through an episode of a simple game: the agent starts at position (0,0). At each step, it can move right (+1 x) or down (+1 y). The goal is to reach (3,3). Rewards are -1 for each step. If the agent moves right, right, right, down, down, down, write the full trajectory including states, actions, and rewards.

Q35. ■ In the above game, what is the return (cumulative discounted reward) from the trajectory you wrote? Assume $\gamma = 0.95$.

Q36. ■ What is the difference between a "step" and an "episode"? How many steps might occur in one episode?

Q37. ■ An agent explores a 4x4 gridworld. On its way to the goal, it takes a wrong path and gets stuck in a loop. The loop lasts 1000 steps. Should the episode end at 1000 steps, or should there be a maximum episode length? Explain your reasoning.

Q38. ■ In a maze environment, the agent can take actions "up", "down", "left", "right". Some paths are dead-ends. Describe how an episode would unfold if the agent wanders into a dead-end. What should happen?

Q39. ■ A robot is learning to walk. Each step of the walking motion is a timestep. An "episode" might be "walk for 30 seconds" or "walk 100 meters". Why is defining an episode boundary important for RL training?

Q40. ■ Consider a continuous control task like balancing a pole. The state is (angle, angular_velocity). At each timestep, the agent applies a force (action). The episode ends if the pole falls (angle > threshold). Write a pseudocode loop that simulates one episode, including state, action, reward, and terminal state checking.

Day 5 — Reward Design and RL Problem Formulation

Learning Objectives

- Understand reward shaping and its implications
- Learn about sparse vs dense rewards
- Design reward functions for complex tasks
- Understand credit assignment

Q41. ■ A robot learns to pick up objects. The designer gives a reward of +1 for picking up the object, and 0 otherwise. This is a "sparse reward". Why might this be hard for the agent to learn from? Suggest a "dense reward" alternative.

Q42. ■ In a game, suppose the agent gets +100 only for winning the game, and 0 for every other action. The game is 1000 timesteps long. How many trajectories might the agent need to see a single +100 reward? What is the learning challenge here?

Q43. ■ Define "credit assignment problem" in RL. Give an example.

Q44. ■ A recommendation agent recommends movies. The user watches 5 movies recommended by the agent. After watching all 5, the user rates the overall experience. How should the agent assign credit to each recommendation?

Q45. ■ Design a reward function for a self-driving car. What should you reward? (safe driving, reaching destination quickly, passenger comfort, etc.) How would you balance these objectives? Would you use rewards of +1, -1, or something else?

Q46. ■ Suppose you design a reward: $R(s, a) = 0$ for all steps except the last, where $R(s_{\text{final}}, a_{\text{final}}) = +100$. This is completely sparse reward. How might an agent learn anything if it randomly explores? (Hint: think about the return only appearing after many random steps.)

Q47. ■ Propose a reward function for a robot learning to cook a meal. Should you reward the robot for each correct step, or only when the meal is done? Why?

Q48. ■ What is "reward shaping"? How might you add intermediate rewards to a sparse-reward task to help learning?

Q49. ■ In a game where the agent needs to collect coins and avoid enemies: Should you give +1 for collecting a coin, +0.5 for avoiding an enemy in a step, or just +1000 for winning? Explain the pros and cons of each approach.

Q50. ■ An agent is learning to write essays. The only reward is 0 or 1 based on whether the essay is "good". What is the problem with this reward structure? How would you redesign it to help the agent learn step-by-step?

Day 6 — Value Functions and Bellman Expectation Equations

Learning Objectives

- Understand state-value function $V(s)$
- Understand action-value function $Q(s,a)$
- Learn the Bellman expectation equation
- Calculate value functions manually

Q51. ■ Define the state-value function $V(s)$. What does $V(s)$ represent? What units does it have?

Q52. ■ Define the action-value function $Q(s, a)$. How is it different from $V(s)$?

Q53. ■ An agent is in state s . From s , it can take actions a_1 or a_2 . Under policy π , $P(a_1|s) = 0.7$ and $P(a_2|s) = 0.3$. If $Q(s, a_1) = 5$ and $Q(s, a_2) = 2$, what is $V(s)$ under this policy?

Q54. ■ State the Bellman expectation equation for $V(s)$. What does it mean intuitively?

Q55. ■ An agent is in state $s_{\blacksquare}$. It takes an action a that leads to reward $r = 3$. The action transitions to $s_{\blacksquare}$ with probability 1.0. $V(s_{\blacksquare}) = 10$ under the policy. $\gamma = 0.9$. Calculate the value $V(s_{\blacksquare})$ using the Bellman equation (assuming the policy always takes this action).

Q56. ■ An action a from state s has two possible outcomes: (1) Transition to s_a with reward 5 (prob 0.6), (2) Transition to s_b with reward -2 (prob 0.4). If $V(s_a) = 20$ and $V(s_b) = 5$, $\gamma = 0.95$, what is $Q(s, a)$?

Q57. ■ Write the Bellman expectation equation for $Q(s, a)$. How does it relate to $V(s)$?

Q58. ■ A state s has three possible actions: a_1, a_2, a_3 with Q -values 10, 5, 8 respectively. A greedy policy always picks the action with the highest Q -value. What is $V(s)$ under this policy?

Q59. ■ Why might a value function be easier to compute than simply simulating all future trajectories? What is the advantage of knowing $V(s)$?

Q60. ■ The Bellman equation uses bootstrapping: $V(s)$ depends on $V(s')$. Explain what bootstrapping means and why it's useful.

Day 7 — Bellman Optimality Equations and Optimal Policies

Learning Objectives

- Understand optimal value functions $V^*(s)$ and $Q^*(s,a)$
- Learn the Bellman optimality equation
- Understand optimal policies
- Calculate optimal values

Q61. ■ Define the optimal state-value function $V^*(s)$. What is the relationship between $V^*(s)$ and $V(s)$?

Q62. ■ State the Bellman optimality equation for $V^*(s)$. How does it differ from the expectation equation?

Q63. ■ An agent in state s can take two actions: a_1 or a_2 . If $Q^*(s, a_1) = 10$ and $Q^*(s, a_2) = 7$, what is $V^*(s)$?

Q64. ■ From state s , action a leads to state s' with probability 1.0, reward 5, and $V^*(s') = 20$. If $\gamma = 0.9$, calculate $Q^*(s, a)$.

Q65. ■ An action a from state s has two outcomes: (1) Transition to s_a with reward 2 (prob 0.8), (2) Transition to s_b with reward 1 (prob 0.2). If $V^*(s_a) = 15$ and $V^*(s_b) = 8$, $\gamma = 0.99$, calculate $Q^*(s, a)$.

Q66. ■ What is an "optimal policy"? How is it related to $V^*(s)$ and $Q^*(s,a)$?

Q67. ■ A state s has available actions with Q^* values: $Q^*(s, a_1)=20$, $Q^*(s, a_2)=15$, $Q^*(s, a_3)=18$. What is $V^*(s)$? What is the optimal policy for this state?

Q68. ■ An agent has learned an optimal policy π^* . At each state, it takes the action that maximizes expected future return. Is this policy deterministic or stochastic? Explain.

Q69. ■ In a 3-state MDP (state A, B, C), suppose $V^*(A) = 50$, $V^*(B) = 30$, $V^*(C) = 10$. If an agent is following the optimal policy, which state would it prefer to start in? Why?

Q70. ■ Write the relationship between $V^*(s)$ and the optimal policy. Given $V^*(s)$ for all states, how would you derive the optimal policy?

Day 8 — Dynamic Programming: Policy Iteration

Learning Objectives

- Understand policy evaluation (prediction)
- Understand policy improvement
- Learn policy iteration algorithm
- Manually perform policy iteration steps

Q71. ■ What is Dynamic Programming? How does it help solve RL problems?

Q72. ■ Explain the two steps of policy iteration: (1) policy evaluation and (2) policy improvement.

Q73. ■ Consider a 2-state MDP. State A and State B. Policy π always takes action "go" from A to B, and "stay" from B. From A→B: reward 10. From B→B: reward 5. $\gamma = 0.9$. Initial $V(A) = 0$, $V(B) = 0$. Perform one iteration of policy evaluation. Calculate new $V(A)$ and $V(B)$.

Q74. ■ In policy improvement, given current value estimates $V(s)$, how do you update the policy? Write the update rule.

Q75. ■ A state s has two actions: a_1 (leads to s with reward 1, prob 1) and a_2 (leads to other state with reward 10, prob 1). Current values: $V(s) = 5$, $V(\text{other}) = 0$. $\gamma = 0.9$. Calculate $Q(s, a_1)$ and $Q(s, a_2)$. Which action would policy improvement select?

Q76. ■ Explain why policy iteration converges. That is, why does repeating policy evaluation and policy improvement eventually lead to an optimal policy?

Q77. ■ After running policy iteration for many iterations, you notice the policy doesn't change, but the values keep increasing. Is this a problem? Explain.

Q78. ■ In a 3-state gridworld, you start with a random policy. After policy evaluation, you get $V(s_1)=10$, $V(s_2)=15$, $V(s_3)=8$. If from s_1 you can go to s_2 or s_3 , and rewards are +5 and +3 respectively ($\gamma=0.9$), which action will policy improvement choose?

Q79. ■ Describe the key difference between policy iteration and value iteration. (Hint: value iteration skips policy evaluation.)

Q80. ■ In policy evaluation, we update $V(s) \leftarrow E[r + \gamma V(s') | \pi]$. If we perform multiple sweeps through all states, when can we stop the evaluation? Explain the convergence condition.

Day 9 — Dynamic Programming: Value Iteration

Learning Objectives

- Learn value iteration algorithm
- Understand Bellman backup operation
- Compare value iteration with policy iteration
- Manually perform value iteration

Q81. ■ State the value iteration update rule. How does it combine policy evaluation and policy improvement?

Q82. ■ A 2-state MDP: state A and B. From A, action "go" leads to B with reward 10 (prob 1). From B, action "stay" leads to B with reward 5 (prob 1). $\gamma = 0.9$. Initial: $V(A) = 0$, $V(B) = 0$. Perform two iterations of value iteration. Calculate $V(A)$ and $V(B)$.

Q83. ■ In value iteration: $V(s) \leftarrow \max_a E[r + \gamma V(s') | s, a]$. Explain why we take the "max" over actions. What does this represent?

Q84. ■ A state has three possible actions with expected returns: 5, 8, 6. Using value iteration, what is the updated $V(s)$?

Q85. ■ In a 3-state gridworld: state A at (0,0), state B at (1,0), state C at (1,1). From A: go right $\rightarrow$ B (reward 1), go down $\rightarrow$ C (reward 0). From B: go left $\rightarrow$ A (reward -5), go down $\rightarrow$ C (reward 2). From C: all actions lead to C (reward 10, terminal). $\gamma = 0.9$. Initial $V(A)=V(B)=V(C)=0$. Perform one iteration of value iteration. Calculate new $V(A)$, $V(B)$, $V(C)$.

Q86. ■ Compare policy iteration and value iteration. Which converges faster in practice? Why?

Q87. ■ Why is value iteration called "asynchronous" dynamic programming? (Hint: think about how it updates values.)

Q88. ■ After running value iteration to convergence, how would you extract the optimal policy? What role does $Q(s, a)$ play?

Q89. ■ In value iteration, you notice that some states' values change very little after each iteration, while others change a lot. Is this normal? Explain.

Q90. ■ A 4x4 gridworld has 16 states. Using value iteration, approximately how many iterations might you need until values converge (assuming random initialization)? Would policy iteration need more or fewer?

Day 10 — Monte Carlo Methods and Episode Returns

Learning Objectives

- Understand Monte Carlo prediction
- Learn first-visit and every-visit MC
- Calculate returns from episodes
- Understand advantages and disadvantages of MC

Q91. ■ What is Monte Carlo in RL? How is it different from dynamic programming?

Q92. ■ Define first-visit MC and every-visit MC. What is the difference?

Q93. ■ An agent collects an episode: $S=[A, B, B, C]$, $A=[\text{go}, \text{go}, \text{end}, \text{N/A}]$, $R=[1, 2, 5]$. $\gamma = 0.9$. Calculate $G(t)$ (return) at each timestep.

Q94. ■ After running first-visit MC for 100 episodes, you have visited state s a total of 15 times (first-visit). The returns from those 15 visits sum to 150. What is $V(s)$ estimate?

Q95. ■ An episode has trajectory with states $[s_1, s_2, s_3, s_4]$ and rewards $[10, 0, 0, 5]$. $\gamma = 0.95$. Calculate the return from each timestep: $G(s_1)$, $G(s_2)$, $G(s_3)$, $G(s_4)$.

Q96. ■ Why does Monte Carlo require complete episodes? Why can't you update values mid-episode?

Q97. ■ Monte Carlo has high variance. Explain what this means and why high variance is a problem for learning.

Q98. ■ An environment has long episodes (1000 timesteps). Is Monte Carlo a good algorithm to use? What is a potential drawback?

Q99. ■ Compare Monte Carlo prediction to dynamic programming prediction. When would you use MC over DP?

Q100. ■ In Monte Carlo, the value estimate $V(s)$ is updated as: $V(s) \leftarrow (V(s) * N + G) / (N+1)$, where N is the number of visits and G is the return. If $V(s) = 10$ after 10 visits (sum 100), and a new episode returns $G=50$, calculate the new $V(s)$.

Day 11 — Temporal Difference Learning and TD Error

Learning Objectives

- Understand TD learning
- Learn TD target and TD error
- Understand bootstrapping
- Calculate TD updates

Q101. ■ What is Temporal Difference (TD) learning? How is it different from Monte Carlo?

Q102. ■ State the TD(0) update rule: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$. Explain each term.

Q103. ■ Define "TD target" and "TD error". What do they represent?

Q104. ■ An agent is in state s with $V(s) = 5$. It takes an action, receives reward $r = 3$, and transitions to s' with $V(s') = 8$. $\alpha = 0.1$, $\gamma = 0.9$. Calculate the TD update and the new $V(s)$.

Q105. ■ TD learning uses "bootstrapping": it updates $V(s)$ using $V(s')$. What is bootstrapping and why is it useful?

Q106. ■ Compare TD and Monte Carlo: Which can update before the episode ends? Which has lower variance? Which requires complete episodes?

Q107. ■ In TD learning, if the TD error is large, what does that mean? Is a large TD error good or bad?

Q108. ■ Two states: s and s' . After observing $r=5$, $\gamma=0.9$, $V(s)=10$, $V(s')=12$, $\alpha=0.2$. Calculate the TD error and the updated $V(s)$.

Q109. ■ What is the learning rate α ? Why is it important? What happens if α is too large? Too small?

Q110. ■ In TD learning, the value estimate $V(s)$ is not the true return. Why do we still use $V(s')$ in the update, even though it might be wrong?

Day 12 — SARSA: On-Policy Temporal Difference Control

Learning Objectives

- Understand on-policy learning
- Learn SARSA algorithm
- Understand ϵ -greedy exploration
- Calculate SARSA updates

Q111. ■ What is the difference between on-policy and off-policy learning?

Q112. ■ State the SARSA update rule: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$. What does each term mean?

Q113. ■ An agent uses SARSA with ϵ -greedy exploration. At each step, with probability ϵ , it takes a random action; otherwise, it takes the greedy action (max Q value). If $\epsilon = 0.1$, what is the probability of exploring? What is the probability of exploiting?

Q114. ■ An agent in state s takes action a (using ϵ -greedy), receives reward $r=2$, and reaches state s' . In s' , the ϵ -greedy policy picks action a' . Given $Q(s, a)=3$, $Q(s', a')=7$, $\alpha=0.1$, $\gamma=0.9$, calculate the SARSA update.

Q115. ■ What is "on-policy" about SARSA? Why is it called that?

Q116. ■ Two scenarios: (1) $\epsilon = 0.5$ (high exploration), (2) $\epsilon = 0.01$ (low exploration). Which would learn faster initially? Which would find a better policy eventually? Explain.

Q117. ■ In SARSA, after taking action a from state s , you observe the next action a' in state s' . Why do you use $Q(s', a')$ instead of $\max Q(s', a)$ in the update?

Q118. ■ An RL agent using SARSA is learning to navigate a cliff. The optimal policy hugs the cliff edge for speed. The agent follows ϵ -greedy with $\epsilon=0.1$. It sometimes falls off the cliff (large negative reward). Does SARSA eventually learn the optimal (risky) policy? Why or why not?

Q119. ■ Calculate a SARSA update: $s="grid1"$, $a="right"$, $r=0$, $s'="grid2"$, $a'="up"$, $Q(s,a)=5$, $Q(s',a')=6$, $\alpha=0.15$, $\gamma=0.95$.

Q120. ■ In SARSA, if $\epsilon \rightarrow 0$ (no exploration), what policy does the agent learn? Is it the optimal policy? Why or why not?

Day 13 — Q-Learning: Off-Policy Control

Learning Objectives

- Understand off-policy learning
- Learn Q-learning algorithm
- Understand behavioral vs target policy
- Calculate Q-learning updates

Q121. ■ What is off-policy learning? How is it different from on-policy learning?

Q122. ■ State the Q-learning update rule: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. What is different from SARSA?

Q123. ■ In Q-learning, the agent follows an ϵ -greedy policy (behavioral policy) to explore. However, the update rule uses $\max Q(s', a')$. This learns an implicit target policy. What is this target policy?

Q124. ■ An agent in state $s="A"$ takes action $a="go"$ (via ϵ -greedy), receives $r=5$, and reaches $s'="B"$. The best action from B has $Q(s', "best")=10$. Other actions from B have Q values: 8, 7. Current $Q(s, a)=3$. $\alpha=0.1$, $\gamma=0.9$. Calculate the Q-learning update.

Q125. ■ Why can Q-learning use ϵ -greedy for exploration but still learn an optimal deterministic policy?

Q126. ■ SARSA vs Q-Learning: An agent takes a risky action a in state s (via ϵ -exploration). The action leads to a bad outcome and a large negative reward. Under SARSA, how is $Q(s, a)$ updated vs Q-learning? Which would penalize the action more?

Q127. ■ A cliff-walking task: the optimal policy takes risky shortcuts; a safer policy exists. Will SARSA or Q-learning learn the risky optimal policy? Explain.

Q128. ■ Calculate a Q-learning update: $s="pos1"$, $a="jump"$, $r=1$, $s'="pos2"$, $\max_{a'} Q(s', a')=8$, $Q(s, a)=2$, $\alpha=0.05$, $\gamma=0.99$.

Q129. ■ In Q-learning, suppose $\max_{a'} Q(s', a')$ is initially overestimated (too high). How does this affect learning? Is this a problem?

Q130. ■ Why is Q-learning guaranteed to converge to the optimal Q^* even though the agent uses ϵ -greedy exploration?

Day 14 — Learning Algorithms: MC, TD, SARSA, Q-Learning Comparison

Learning Objectives

- Compare Monte Carlo, TD, SARSA, Q-Learning
- Understand when to use each algorithm
- Analyze trade-offs
- Solve practical problems with appropriate algorithms

Q131. ■ Create a table comparing Monte Carlo, TD, SARSA, and Q-Learning on these dimensions: on-policy/off-policy, needs full episodes, variance, bias, convergence speed.

Q132. ■ An environment has very long episodes (10,000 timesteps). Which algorithm would you prefer: Monte Carlo or TD? Why?

Q133. ■ In a game-playing RL agent, should you use SARSA or Q-Learning? What is the key difference in what they would learn?

Q134. ■ An agent uses Monte Carlo with first-visit updates. After 1000 episodes, it has visited state $s=100$ times and the sum of returns is 500. What is $V(s)$? Now another episode returns $G(s)=60$. What is the new $V(s)$?

Q135. ■ An agent learns Q-values using Q-learning in a dangerous environment. Occasionally, it encounters a very negative reward (-100). How does this affect the Q-values? Is the agent learning to avoid these situations?

Q136. ■ Propose an RL algorithm for an autonomous vehicle learning to drive in traffic: Should it use Monte Carlo, TD, SARSA, or Q-Learning? Justify your choice.

Q137. ■ TD learning can update mid-episode, but Monte Carlo cannot. Why is the ability to update mid-episode valuable?

Q138. ■ An agent uses Q-learning in a deterministic environment. Why might Q-learning still converge even though the environment is deterministic?

Q139. ■ Describe a scenario where SARSA would outperform Q-Learning, and vice versa.

Q140. ■ In Q-learning vs SARSA, which has lower bias? Lower variance? What does this imply about sample efficiency and stability?

Day 15 — Exploration vs Exploitation and ϵ -Greedy Strategy

Learning Objectives

- Understand exploration-exploitation trade-off
- Learn ϵ -greedy and epsilon decay
- Understand exploration methods
- Design exploration schedules

Q141. ■ Define "exploitation" and "exploration" in RL. Give an example of each.

Q142. ■ An agent has learned Q-values for all state-action pairs. To maximize immediate reward, it should always pick the action with the highest Q-value. Is this optimal in the long run? Why or why not?

Q143. ■ Explain the ϵ -greedy strategy. If $\epsilon=0.1$, what is the probability of exploring vs exploiting?

Q144. ■ An agent uses ϵ -greedy with $\epsilon=0.05$. At each timestep, what happens? List the steps (random action, greedy action, probabilities).

Q145. ■ Why might you want to decay ϵ over time? That is, why start with high ϵ and gradually decrease it?

Q146. ■ An agent learns for 100,000 timesteps. ϵ starts at 1.0 and decays linearly to 0.01. Design an epsilon decay schedule: write $\epsilon(t)$ as a function of timestep t .

Q147. ■ An agent uses $\epsilon=0$ (no exploration) from the start. What might happen? Will it learn anything?

Q148. ■ An agent uses $\epsilon=1.0$ forever (always random). Will it learn the optimal policy?

Q149. ■ An RL agent for a recommendation system uses ϵ -greedy exploration. High exploration (high ϵ) might show users unfamiliar items. High exploitation (low ϵ) might bore users. How would you balance ϵ ?

Q150. ■ An agent learns in an environment with 100 possible actions. If $\epsilon=0.01$, how many timesteps until the agent is expected to explore once? Is this good for learning?

Day 16 — Multi-Armed Bandits and Bandit Algorithms

Learning Objectives

- Understand the multi-armed bandit problem
- Learn greedy and ϵ -greedy for bandits
- Understand regret
- Analyze bandit algorithms

Q151. ■ What is the multi-armed bandit (MAB) problem? How is it different from the full RL problem?

Q152. ■ A bandit has 3 arms with unknown reward distributions. Arm 1 has mean reward 5, Arm 2 has mean 7, Arm 3 has mean 3. If you had to choose one arm forever, which would you pick? But if you don't know the means, what is your strategy?

Q153. ■ Define "regret" in bandits. Why does ϵ -greedy have growing regret?

Q154. ■ A bandit has two arms. You pull Arm 1 ten times and get rewards: 3, 4, 2, 5, 3 (then two pulls). Actually, you pulled: 3, 4, 2, 5, 3, 1, 2. You pull Arm 2 five times: 6, 7, 6, 5, 6. Using sample average, estimate the mean reward for each arm.

Q155. ■ An ϵ -greedy bandit agent with $\epsilon=0.1$ runs for 1000 pulls. How many times (approximately) does it explore? How many times exploit?

Q156. ■ Upper Confidence Bound (UCB) is a bandit algorithm that balances exploration and exploitation using confidence intervals. Why might UCB be better than ϵ -greedy?

Q157. ■ A bandit with $K=4$ arms runs for $N=1000$ pulls. You estimate the mean of each arm using sample average. After convergence, which arm will be pulled most often? Why?

Q158. ■ An optimistic initialization bandit starts with high estimated rewards for all arms. Why might this encourage exploration?

Q159. ■ Compare greedy ($\epsilon=0$), ϵ -greedy ($\epsilon=0.1$), and random ($\epsilon=1.0$) bandit strategies. Draw or describe their learning curves: regret vs pulls.

Q160. ■ A bandit application: a website tests two designs (A/B testing). Design A converts 10% of users, Design B converts 11%. You don't know these values. Using a bandit algorithm, how would you allocate traffic to each design?

Day 17 — Tabular Q-Learning Implementation

Learning Objectives

- Understand Q-tables
- Implement Q-learning step-by-step
- Debug Q-learning code
- Analyze Q-table learning

Q161. ■ What is a Q-table? Describe its structure (rows, columns, values).

Q162. ■ A gridworld has 5x5 cells (25 states) and 4 actions (up, down, left, right). Create a Q-table structure. What are its dimensions? What does each entry represent?

Q163. ■ Write pseudocode for the Q-learning training loop: initialization, episode loop, step loop, observation (s, a, r, s'), Q-update, next state transition.

Q164. ■ In Python, create a Q-table for a 3x3 gridworld with 4 actions. Initialize all Q-values to 0. Write code to access and update Q(state, action).

Q165. ■ Implement ϵ -greedy action selection in Python: given a Q-table, epsilon, and a state, select an action.

Q166. ■ Write Python code for one Q-learning update step: given Q, state, action, reward, next_state, alpha, gamma, compute and apply the update.

Q167. ■ A Q-learning agent runs for 1000 episodes in a simple gridworld. The Q-table has been populated with estimates. How would you extract the learned policy? Write code.

Q168. ■ A Q-learning agent gets stuck: it repeatedly visits the same states without reaching the goal. The Q-values are not converging. What might be wrong? (Hint: think about exploration, learning rate, reward structure.)

Q169. ■ In a Q-table, a state-action pair (s, a) has been visited 1000 times. The Q-value still changes significantly with each update. What does this suggest about α ? Should α be larger or smaller?

Q170. ■ Implement a monitoring system for Q-learning: track the average Q-value across the table over time. Write code to log and visualize this (pseudocode is fine).

Day 18 — GridWorld Environments and Custom Environments

Learning Objectives

- Understand GridWorld structure
- Design environments for RL
- Implement GridWorld dynamics
- Test RL agents in GridWorlds

Q171. ■ What is a GridWorld environment? Describe the state space, action space, and reward structure.

Q172. ■ Design a 4x4 GridWorld: Agent starts at (0,0), goal is at (3,3), obstacles at (1,1) and (2,2). Actions: up, down, left, right. Rewards: +1 for reaching goal, -1 for each step, -10 for hitting obstacles. Write the environment dynamics.

Q173. ■ In a GridWorld, the agent tries to move up but the grid wraps around (toroidal). If at (0, 2) and moves up, it appears at (3, 2) in a 4x4 grid. Write the state transition function.

Q174. ■ Implement a GridWorld step function in pseudocode: inputs are state and action, outputs are next_state and reward.

Q175. ■ A GridWorld has "slippery floors": with probability 0.1, the agent moves in a random direction instead of the intended one. How would you implement this stochasticity?

Q176. ■ Design a GridWorld where the goal location changes each episode. How would this affect RL learning? Would Q-learning still work?

Q177. ■ Extend a GridWorld with "enemies" that move randomly. If the agent collides with an enemy, the episode ends. How would you change the reward structure?

Q178. ■ A GridWorld has 16x16 cells. Using tabular Q-learning, how many Q-table entries are needed? Is tabular RL practical for this size?

Q179. ■ Design an "escape room" GridWorld: the agent must collect keys in a specific order to unlock doors. Describe the state space and reward function.

Q180. ■ Implement a rendering function for a GridWorld that displays the agent, obstacles, and goal as ASCII. Pseudocode is acceptable.

Day 19 — Debugging and Improving RL Agents

Learning Objectives

- Identify common RL learning failures
- Debug Q-learning implementations
- Improve agent performance
- Analyze learning curves

Q181. ■ A Q-learning agent is training but the rewards are not improving. What are the top 5 things you would check?

Q182. ■ An agent's Q-values grow unbounded ($\rightarrow \infty$). What is likely wrong? (Hint: think about the reward structure or learning rate.)

Q183. ■ An agent learns well at first but then performance plateaus and doesn't improve. What might be the cause? (Hint: think about exploration, learning rate, or reward design.)

Q184. ■ A Q-learning agent is given this update rule (WRONG): $Q(s,a) \leftarrow Q(s,a) * (1-\alpha) + \alpha * (r + \gamma * Q(s', a'))$. What is wrong? How should it be?

Q185. ■ An agent explores very little (ϵ is too small). Its performance improves quickly but plateaus at a suboptimal level. Explain what is happening.

Q186. ■ A debugging scenario: an agent reaches the goal every episode but still gets negative total reward. Why might this be? How would you fix it?

Q187. ■ Write code to plot a learning curve: average reward per episode vs episode number. How would you interpret the shape of this curve to diagnose learning problems?

Q188. ■ An agent's policy is deterministic (always takes the same action in each state). The policy is optimal for the current Q-values. But the policy never reaches the goal. What is the problem?

Q189. ■ A Q-learning implementation has this bug: it uses the current best action for the update, not the sampled action. What algorithm is this? Is it Q-learning or something else?

Q190. ■ Design a test suite for a Q-learning agent: what experiments would you run to verify that the agent is learning correctly?

Day 20 — Advanced Exploration: UCB, Thompson Sampling, and Curiosity

Learning Objectives

- Understand Upper Confidence Bound (UCB)
- Learn Thompson Sampling
- Understand intrinsic motivation and curiosity
- Compare exploration techniques

Q191. ■ Upper Confidence Bound (UCB) balances exploitation and exploration using confidence intervals. Explain the intuition: which arms should be explored more?

Q192. ■ In UCB1, the action selection rule is: $a = \operatorname{argmax}_a [Q(a) + c * \sqrt{\ln(N) / N(a)}]$. Explain each term. What happens if an arm is rarely selected?

Q193. ■ Thompson Sampling uses a probabilistic approach to exploration: sample Q-values from a posterior distribution and pick the action with the highest sample. Why might this work better than ϵ -greedy?

Q194. ■ A bandit arm has been pulled 100 times with sample mean 5 and standard deviation 2. Using UCB with $c=1$, calculate the upper confidence bound for this arm.

Q195. ■ Compare ϵ -greedy, UCB, and Thompson Sampling: Which explores the most? Which is most data-efficient? Which is easiest to implement?

Q196. ■ Curiosity-driven exploration: an agent is rewarded for visiting novel states (low visitation count). How might this help find better policies? What are the drawbacks?

Q197. ■ In a complex environment, pure ϵ -greedy exploration might not find interesting regions of the state space. How could you use curiosity or count-based exploration to help?

Q198. ■ An agent uses UCB exploration in a non-stationary environment (reward distributions change over time). Will UCB still work? Why or why not?

Q199. ■ Design a hybrid exploration strategy: combine ϵ -greedy with count-based bonuses (extra reward for visiting new states). Write pseudocode.

Q200. ■ An exploration strategy uses: $\pi(a|s) \propto \exp(Q(a) / \tau)$, where τ is temperature. If τ is large, what happens? If τ is small? How is this related to exploration?

Day 21 — Probability, Statistics, and Expected Values

Learning Objectives

- Understand probability basics
- Learn expected value
- Understand random variables and distributions
- Apply to RL contexts

Q201. ■ Define probability. If an event has probability 0.3, what does that mean?

Q202. ■ What is a random variable? Give an example from RL.

Q203. ■ An RL agent receives rewards following a distribution: $P(r=1)=0.5$, $P(r=2)=0.3$, $P(r=5)=0.2$. Calculate the expected reward.

Q204. ■ Define conditional probability $P(A|B)$. In RL, what does $P(s'|s, a)$ represent?

Q205. ■ An action a from state s has outcomes: (1) s' , reward 10 (prob 0.7), (2) s'' , reward 2 (prob 0.3). Calculate $E[r | s, a]$ (expected immediate reward).

Q206. ■ A continuous random variable X has probability density $f(x) = 0.5$ for $x \in [0, 2]$. Calculate $E[X]$.

Q207. ■ Explain variance in a distribution. Why might an RL agent prefer low-variance strategies?

Q208. ■ A policy $\pi(a|s)$ is stochastic. Calculate $E_{a \sim \pi}[Q(s,a)]$, the expected Q-value under the policy.

Q209. ■ Define standard deviation and coefficient of variation. Why are these useful for analyzing RL algorithms?

Q210. ■ An RL agent's reward has mean 5 and standard deviation 2. Calculate the probability that the reward is between 3 and 7 (assuming Normal distribution).

Day 22 — Linear Algebra: Vectors, Matrices, and Operations

Learning Objectives

- Understand vectors and matrices
- Learn matrix operations
- Understand transformations and dimensions
- Apply to neural networks

Q211. ■ What is a vector? Write an example. What is its dimension?

Q212. ■ What is a matrix? Write an example. What are its dimensions?

Q213. ■ Given vectors $u = [1, 2, 3]$ and $v = [4, 5, 6]$, calculate the dot product $u \cdot v$.

Q214. ■ A matrix W has shape $(5, 3)$. A vector x has shape $(3,)$. What is the shape of $W @ x$ (matrix-vector product)?

Q215. ■ Two matrices: A (shape 2×3) and B (shape 3×4). What is the shape of $A @ B$?

Q216. ■ In a neural network, the input layer has 10 neurons, hidden layer has 5 neurons. What is the shape of the weight matrix W from input to hidden?

Q217. ■ Calculate: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Compute $A @ B$.

Q218. ■ A weight matrix W has shape $(5, 3)$. A batch of inputs X has shape $(\text{batch_size}, 3)$. What is the shape of $X @ W$?

Q219. ■ In deep RL, states are vectors. Weights are matrices. Output Q-values are scalars for each action. Describe the matrix operations in a simple network: input $\rightarrow$ hidden $\rightarrow$ output.

Q220. ■ What is the transpose of a matrix? Given W (shape 4×3), what is $W.T$? What is $(W.T).T$?

Day 23 — Neural Networks and Value Function Approximation

Learning Objectives

- Understand neural network basics
- Learn function approximation
- Understand why function approximation is needed
- Learn gradient descent and backpropagation intuition

Q221. ■ What is a neural network? Describe a simple network: input layer, hidden layer, output layer.

Q222. ■ In a Q-network, the input is a state (vector) and the output is Q-values for each action. Design the network architecture (layers, sizes).

Q223. ■ What is "function approximation"? Why do we need function approximation in RL?

Q224. ■ A tabular Q-learner handles 1000 states and 4 actions: how many Q-values to store? A neural network Q-learner has 1000 weights: which is more efficient?

Q225. ■ Explain the forward pass in a neural network. Given input x , weights $W1$, $W2$, and biases $b1$, $b2$, write the forward pass.

Q226. ■ What is an activation function? Why do we use ReLU instead of linear activation?

Q227. ■ In supervised learning, a network minimizes loss: $L = (y_{\text{true}} - y_{\text{pred}})^2$. In RL Q-learning with function approximation, what is the "loss" or "TD error"?

Q228. ■ Explain gradient descent: what is the gradient? What does it tell us? How do we use it to update weights?

Q229. ■ A weight w is updated as: $w \leftarrow w - \alpha * dL/dw$. If dL/dw is positive, does w increase or decrease? Is this the direction of decreasing loss?

Q230. ■ Explain the intuition of backpropagation (you do not need math, just intuition). How does the network "learn" which weights to adjust?

Day 24 — Deep Q-Networks: From Tabular to Deep RL

Learning Objectives

- Understand DQN architecture
- Learn experience replay
- Learn target networks
- Understand DQN improvements

Q231. ■ What is a Deep Q-Network (DQN)? How does it extend tabular Q-learning?

Q232. ■ In DQN, the network takes state s as input and outputs Q-values for all actions. What is the network architecture (input size, output size, hidden layers)?

Q233. ■ DQN uses experience replay: it stores transitions (s, a, r, s') in a buffer and samples batches. Why is experience replay useful?

Q234. ■ Write the DQN loss function: $L = (r + \gamma * \max_{a'} Q_target(s', a') - Q(s, a))^2$. Explain each term.

Q235. ■ DQN uses two networks: online and target. Why use two networks instead of one?

Q236. ■ The target network is updated periodically (e.g., every C steps). What would happen if the target network was updated every step? What if it was never updated?

Q237. ■ A DQN training loop: 1. Observe state s , select action a (ϵ -greedy), 2. Execute, get r and s' , 3. Store in replay buffer, 4. Sample batch from buffer, 5. Compute TD target, 6. Compute loss and backprop. Write pseudocode for steps 4-6.

Q238. ■ Double DQN uses: $r + \gamma * Q_target(s', \operatorname{argmax}_a Q_online(s', a))$. Why is this better than standard DQN?

Q239. ■ In DQN, the network might overestimate Q-values for unseen state-action pairs. What is the problem with this? How might it affect learning?

Q240. ■ Design a DQN agent for a game like Atari Pong: What is the input? What is the output? What would be good hyperparameters (learning rate, γ , ϵ -decay)?

Day 25 — Policy Gradients and REINFORCE Algorithm

Learning Objectives

- Understand policy parameterization
- Learn the policy gradient theorem
- Learn REINFORCE algorithm
- Calculate policy gradient updates

Q241. ■ What is a parameterized policy $\pi_{\theta}(a|s)$? How is it different from a policy defined by Q-values?

Q242. ■ The policy gradient theorem states: $\nabla J(\theta) \propto E[\nabla \log \pi_{\theta}(a|s) * Q(s,a)]$. What does this mean? How do we use it to improve the policy?

Q243. ■ REINFORCE uses: $\theta \leftarrow \theta + \alpha * \nabla \log \pi_{\theta}(a|s) * G(t)$. What is $G(t)$? Why use return instead of reward?

Q244. ■ In REINFORCE, if the return G is positive, what happens to $\log \pi_{\theta}(a|s)$? Does $\pi_{\theta}(a|s)$ increase or decrease?

Q245. ■ A simple policy is: $\pi_{\theta}(a|s) = \text{softmax}(\theta_a)$, where θ_a is a parameter for each action. In this case, $\nabla \log \pi_{\theta}(a|s) = 1_a - \pi_{\theta}(a|s)$, where 1_a is 1 for chosen action, 0 elsewhere. Explain the intuition of this gradient.

Q246. ■ REINFORCE has high variance. Explain why. How might you reduce variance without changing the algorithm significantly?

Q247. ■ A common variance-reduction technique is to subtract a baseline $b(s)$ from the return: $\theta \leftarrow \theta + \alpha * \nabla \log \pi_{\theta}(a|s) * (G(t) - b(s))$. What is $b(s)$? What should it be?

Q248. ■ Write pseudocode for REINFORCE training: collect an episode, compute returns, update policy parameters.

Q249. ■ REINFORCE is on-policy: it learns from the trajectory it collected. Can REINFORCE learn from old trajectories (like DQN's experience replay)? Why or why not?

Q250. ■ An advantage of policy gradients over Q-learning: they directly optimize the policy. An advantage of Q-learning: it's off-policy. Which is better? Explain trade-offs.

Day 26 — Actor-Critic Architecture

Learning Objectives

- Understand actor-critic structure
- Learn advantage estimation
- Understand TD error as advantage
- Analyze actor-critic algorithms

Q251. ■ What is an actor-critic method? Describe the two components: actor and critic.

Q252. ■ In actor-critic, the critic estimates $V(s)$. The actor uses this to learn the policy. How does the critic help the actor? (Hint: think about variance reduction.)

Q253. ■ Define "advantage": $A(s, a) = Q(s, a) - V(s)$. What does it represent? Why is it useful in actor-critic?

Q254. ■ The TD error $\delta = r + \gamma V(s') - V(s)$. Explain why the TD error can be used as an estimate of the advantage.

Q255. ■ Write the updates for actor and critic in A2C (Advantage Actor-Critic): Actor: $\theta \leftarrow \theta + \alpha * \nabla \log \pi_{\theta}(a|s) * \text{advantage}$. Critic: $\phi \leftarrow \phi - \beta * (r + \gamma V_{\phi}(s') - V_{\phi}(s))^2$ (MSE loss). Explain each update.

Q256. ■ An actor-critic agent in a continuous control task (robot arm). The actor outputs actions (continuous), the critic outputs state values. How would you sample actions from a continuous policy?

Q257. ■ Compare REINFORCE with baseline (critic) to A2C. What is the key difference?

Q258. ■ In actor-critic, if the critic is poorly trained (bad value estimates), how does this affect the actor's learning? What might happen?

Q259. ■ Asynchronous Advantage Actor-Critic (A3C) runs multiple agents in parallel. Why might parallelization help RL training?

Q260. ■ Design a continuous-control actor-critic agent for a task like "balance a pole": What are the input/output dimensions? What are the network architectures for actor and critic?

Day 27 — Proximal Policy Optimization (PPO)

Learning Objectives

- Understand trust region methods
- Learn PPO objective and clipping
- Understand policy updates
- Analyze PPO stability

Q261. ■ What is PPO? Why was it developed? What problems does it solve?

Q262. ■ The probability ratio: $r_t(\theta) = \pi_\theta(a|s) / \pi_{\theta_{\text{old}}}(a|s)$. What does r_t represent? What happens if $r_t \gg 1$?

Q263. ■ PPO uses clipping: $L = \min(r_t * A, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) * A)$. Explain what the clip function does. Why is clipping useful?

Q264. ■ In PPO, the advantage A can be estimated using GAE (Generalized Advantage Estimation) or simply discounted returns. What are the trade-offs?

Q265. ■ A policy update in PPO: advantages are positive (good action). The ratio $r_t = 2$ (new policy is twice as likely). With $\epsilon = 0.2$, what is the clipped ratio? Is the gradient reduced compared to unclipped?

Q266. ■ PPO typically runs K epochs of updates on the same batch of data. Why use multiple epochs? What could go wrong if K is too large?

Q267. ■ Write pseudocode for a PPO training step: collect trajectories, compute returns/advantages, update policy and value network for K epochs.

Q268. ■ PPO is more stable than basic policy gradients. Why? (Hint: think about the clipping mechanism and trust region.)

Q269. ■ In a PPO implementation, if the clipping threshold ϵ is too small, what might happen? Too large?

Q270. ■ Design a PPO agent for a complex task like "humanoid locomotion" (2D robot walking). Describe hyperparameters you would use and how you would structure the network.

Day 28 — Advanced Algorithms: Rainbow, SAC, TD3, and More

Learning Objectives

- Learn about algorithm variants and improvements
- Understand continuous control methods
- Learn entropy regularization
- Analyze trade-offs between algorithms

Q271. ■ What is Rainbow DQN? What improvements does it combine?

Q272. ■ Dueling DQN splits the Q-network into value and advantage streams: $Q(s,a) = V(s) + (A(s,a) - \text{mean}_a A(s, a'))$. Why does this architecture help learning?

Q273. ■ Prioritized Experience Replay: instead of sampling uniformly from the buffer, sample transitions with high TD error more frequently. Why would this improve learning?

Q274. ■ Distributional RL learns the distribution of returns, not just the mean. Why might learning the distribution be better than learning the mean?

Q275. ■ What is entropy regularization? A policy with high entropy explores more. Why might you want to maximize entropy? What is the trade-off?

Q276. ■ SAC (Soft Actor-Critic) uses: $\text{objective} = E[Q(s,a)] - \alpha H(\pi(.|s))$. What does the second term represent? What happens if α is large?

Q277. ■ TD3 (Twin Delayed DDPG) uses two Q-networks to reduce overestimation. Write the update: $Q = \min(Q1_target, Q2_target)$. Why use the minimum?

Q278. ■ DDPG (Deep Deterministic Policy Gradient) is an off-policy actor-critic for continuous control. How does it differ from A3C? (Hint: on-policy vs off-policy.)

Q279. ■ Curiosity-driven learning adds an intrinsic reward based on prediction error: $r_intrinsic = \text{prediction_error}$. When would you use curiosity-driven learning? What are its benefits and drawbacks?

Q280. ■ Compare value-based (DQN) vs policy-gradient (PPO) vs actor-critic (A3C, SAC) methods: Pros, cons, and when to use each.

Day 29 — Comprehensive Algorithm Analysis and Selection

Learning Objectives

- Compare major RL algorithms
- Reason about algorithm selection
- Analyze trade-offs
- Design solutions for complex problems

Q281. ■ Create a comprehensive comparison table of: DP, Monte Carlo, TD, SARSA, Q-Learning, DQN, REINFORCE, A3C, PPO. Include: on/off-policy, model-free/based, tabular/function approx, sample efficiency, stability, convergence.

Q282. ■ A robot learns to pick up objects from a table. The task has sparse rewards (only when object is picked). Which algorithm would you choose? (Monte Carlo, Q-Learning, DQN, or PPO?) Justify.

Q283. ■ An Atari game has high-dimensional visual observations (images). Would you use tabular RL or deep RL? Which specific algorithm? Why?

Q284. ■ A recommendation system needs to balance exploring new recommendations with exploiting known preferences. Which algorithm (bandit-based, Q-Learning, or policy gradient) would you use? Why?

Q285. ■ An autonomous vehicle learns to drive. The environment is dangerous and non-stationary. Should you use on-policy (SARSA, PPO) or off-policy (Q-Learning, DQN)? Why?

Q286. ■ Compare model-free RL (Q-Learning, PPO) vs model-based RL (planning with a learned model). What are the trade-offs? When would you use each?

Q287. ■ A task has continuous actions (e.g., robot joint angles). Would you use discrete-action algorithms (DQN) or continuous-action (PPO, DDPG)? How would you adapt discrete methods?

Q288. ■ An agent learns in a stochastic environment (randomness in transitions and rewards). Which algorithms handle stochasticity well? Which struggle? Why?

Q289. ■ You have limited computational resources. Should you use simple Q-Learning or complex DQN with experience replay? What are the trade-offs?

Q290. ■ Imagine you are developing an RL system for a real-world application (choose one: robotics, trading, recommendation). Describe the problem, justify your algorithm choice, and

discuss hyperparameters.

Day 30 — Final RL Mastery Assessment

Learning Objectives

- Synthesize all RL concepts
- Solve complex problems
- Reason about algorithm behavior
- Design RL solutions

Q291. ■ A robot learns to navigate a 10x10 maze from start to goal. The maze is known (no randomness). Actions: up, down, left, right. Reward: -1 per step, +100 for goal. (a) Design the state space. (b) Calculate the optimal return from (5,5) to goal at (9,9). (c) Would you use tabular Q-Learning or DQN? Justify.

Q292. ■ An agent learns a policy π with Q-values: $Q(s, a_1)=10$, $Q(s, a_2)=8$, $Q(s, a_3)=5$. Using ϵ -greedy with $\epsilon=0.1$, calculate $P(\text{each action})$ from state s . Now perform one Q-learning update: action a_1 taken, reward $r=2$, next state s' reached, $\max_{a'} Q(s', a')=12$. $\alpha=0.1$, $\gamma=0.9$. Calculate new $Q(s, a_1)$.

Q293. ■ Implement (in pseudocode) a complete Q-learning training loop for GridWorld: initialization, episode loop with max 100 steps, ϵ -greedy selection, Q-update, episode termination condition. Include monitoring: print avg reward every 100 episodes.

Q294. ■ An agent collects three episodes with returns: $G_1=[1+0.9*2+0.81*3]=5.39$, $G_2=[5]$, $G_3=[-2]$. $V(s)$ was initialized to 0. After these three first-visit MC episodes, calculate new $V(s)$.

Q295. ■ DQN with two networks: online and target. Explain when and why the target network is updated. Write the loss: $L = [r + \gamma * \max_{a'} Q_{\text{target}}(s', a') - Q_{\text{online}}(s, a)]^2$. How is this loss backpropagated to update Q_{online} ?

Q296. ■ Policy gradient REINFORCE: Derive why $\theta \leftarrow \theta + \alpha * \nabla \log \pi_{\theta}(a|s) * G(t)$ increases the probability of action a when $G(t) > 0$. Explain the sign of the gradient.

Q297. ■ An actor-critic agent: actor learns $\pi_{\theta}(a|s)$, critic learns $V_{\phi}(s)$. Advantage: $A = r + \gamma V_{\phi}(s') - V_{\phi}(s)$. If $A > 0$, should the actor increase or decrease $\log \pi_{\theta}(a|s)$? If $A < 0$? Explain and write the updates.

Q298. ■ Design an RL solution for a real-world problem of your choice (e.g., website optimization, inventory management, or route planning). Describe: state space, action space, reward function, suitable algorithm, and why.

Q299. ■ An agent trained with Q-learning in environment A is transferred to similar environment B. Its Q-values are initialized from A. Environment B has slightly different dynamics. How would the agent adapt? Would it relearn or leverage previous Q-values? What problems might arise?

Q300. ■ Reflect on your journey through 300 RL questions: (a) Describe the progression from tabular to deep RL in your own words. (b) Which concept was most challenging? (c) Which algorithm surprised you the most? (d) How would you continue learning RL after this workbook?